# Experiment 1 Code:

## MyRemoteClass.java

```
public class MyRemoteClass extends
java.rmi.server.UnicastRemoteObject implements MyInterface {
   public MyRemoteClass() throws java.rmi.RemoteException {
      super();
   }
   public String sayHello() throws java.rmi.RemoteException {
      return "Hello, world!";
   }
}
```

## MyInterface.java

```
public interface MyInterface extends java.rmi.Remote {
   public String sayHello() throws java.rmi.RemoteException;
}
```

## MyClient.java

```
public class MyClient {
   public static void main(String[] args) {
      try {
         MyInterface obj = (MyInterface)
java.rmi.Naming.lookup("//localhost/MyRemoteClass");
         System.out.println(obj.sayHello());
```

```
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

MyServer.java

```
public class MyServer {
    public static void main(String[] args) {
        try {
            MyInterface obj = new MyRemoteClass();
            java.rmi.registry.LocateRegistry.createRegistry(1099);
            java.rmi.Naming.rebind("//localhost/MyRemoteClass", obj);
            System.out.println("MyRemoteClass bound in registry");
        } catch (Exception e) {
            System.err.println("MyRemoteClass exception:");
            e.printStackTrace();
        }
    }
}
```

# Output Screenshots:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL
PS D:\Distributed System Practicals\Practical 1> javac MyInterface.java
PS D:\Distributed System Practicals\Practical 1> javac MyRemoteClass.java
PS D:\Distributed System Practicals\Practical 1> javac MyClient.java
PS D:\Distributed System Practicals\Practical 1> javac MyServer.java
PS D:\Distributed System Practicals\Practical 1> java MyServer
MyRemoteClass bound in registry
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL
PS D:\Distributed System Practicals\Practical 1> java MyClient
Hello, world!
PS D:\Distributed System Practicals\Practical 1>
```

# Experiment 2 Code:

## Hello_World.java

```
import HelloWorldApp.HelloWorld;
import HelloWorldApp.HelloWorldHelper;
import org.omg.CosNaming.*;
import org.omg.CORBA.*;
import org.omg.PortableServer.*;

public class HelloWorldApp {
    public static void main(String[] args) {
        try {
            // Initialize the ORB
            ORB orb = ORB.init(args, null);

            // Create and activate the root POA
            POA rootPOA =
POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
            rootPOA.the_POAManager().activate();

            // Create the HelloWorld servant
            HelloWorldImpl helloWorldImpl = new HelloWorldImpl();

            // Get the object reference for the servant
            org.omg.CORBA.Object ref =
rootPOA.servant_to_reference(helloWorldImpl);
            HelloWorld helloWorldRef = HelloWorldHelper.narrow(ref);

            // Get the root naming context
            org.omg.CORBA.Object objRef =
orb.resolve_initial_references("NameService");
```

```
            NamingContextExt ncRef =
NamingContextExtHelper.narrow(objRef);

            // Bind the object reference to the naming context
            NameComponent[] path = ncRef.to_name("HelloWorld");
            ncRef.rebind(path, helloWorldRef);

            // Run the ORB event loop
            orb.run();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

class HelloWorldImpl extends HelloWorldPOA {
    public String sayHello() {
        return "Hello, world!";
    }
}

class HelloWorldClient {
    public static void main(String[] args) {
        try {
            // Initialize the ORB
            ORB orb = ORB.init(args, null);

            // Get the root naming context
            org.omg.CORBA.Object objRef =
orb.resolve_initial_references("NameService");
            NamingContextExt ncRef =
NamingContextExtHelper.narrow(objRef);

            // Resolve the object reference from the naming context
```

```
            HelloWorld helloWorldRef =
HelloWorldHelper.narrow(ncRef.resolve_str("HelloWorld"));

            // Invoke the remote method
            String message = helloWorldRef.sayHello();
            System.out.println("Server says: " + message);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

# Experiment 3 Code:

## ArraySumMPI.java

```
import mpi.MPI;
public class ArraySumMPI {
    public static void main(String[] args) {
        MPI.Init(args);
        int rank = MPI.COMM_WORLD.Rank();
        int size = MPI.COMM_WORLD.Size();
        int[] array = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; // Sample array
        int n = array.length;
        int localSum = 0;
        int globalSum = 0;
        // Calculate the local sum
        int startIndex = rank * (n / size);
        int endIndex = (rank == size - 1) ? (n - 1) : (startIndex + (n / size) - 1);
        for (int i = startIndex; i <= endIndex; i++) {
            localSum += array[i];
        }
        // Compute the global sum by reducing local sums
        MPI.COMM_WORLD.Reduce(new int[]{localSum}, 0, new
int[]{globalSum}, 0, 1, MPI.INT, MPI.SUM, 0);
        if (rank == 0) {
            System.out.println("Global sum: " + globalSum);
        }

        MPI.Finalize();
    }
}
```

# Experiment 4 Code:

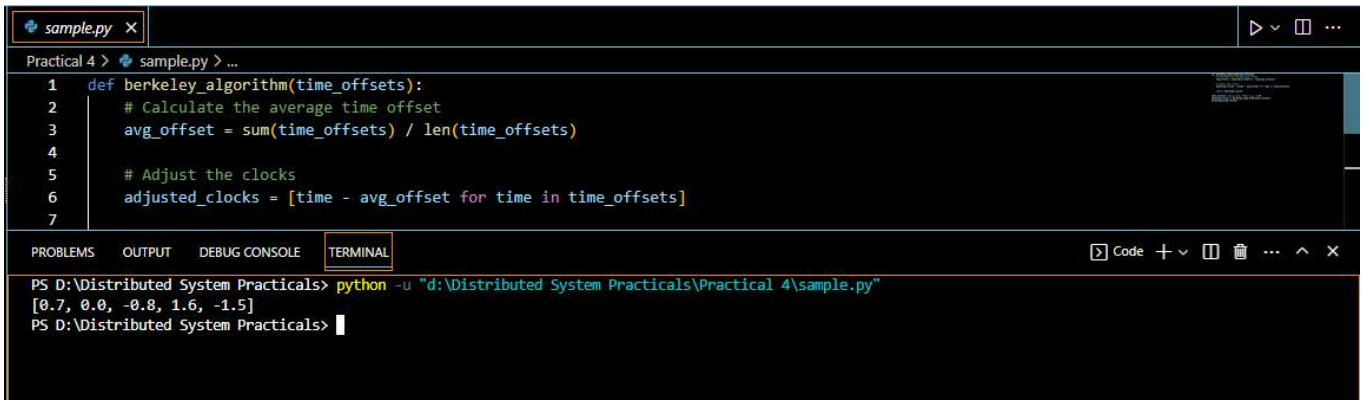
## berkeley_algorithm.py

```
def berkeley_algorithm(time_offsets):
    # Calculate the average time offset
    avg_offset = sum(time_offsets) / len(time_offsets)

    # Adjust the clocks
    adjusted_clocks = [time - avg_offset for time in time_offsets]

    return adjusted_clocks

time_offsets = [1.2, 0.5, -0.3, 2.1, -1.0]
adjusted_clocks = berkeley_algorithm(time_offsets)
print(adjusted_clocks)
```



# Experiment 5 Code:

## token_ring.py

```
class TokenRing:
    def __init__(self, num_nodes):
        self.num_nodes = num_nodes
        self.nodes = [None] * num_nodes

    def set_node(self, node_id, data):
        self.nodes[node_id] = data

    def run(self):
        token = self.nodes
        current_node = 0

        while True:
            # Check if the token is received
            if token:
                print(f"Node {current_node}: Received token with data '{token}'")

            # Process the token or perform some action
            # In this example, we print the data of the current node
            print(f"Node {current_node}: Processing data
'{self.nodes[current_node]}'")

            # Pass the token to the next node
            next_node = (current_node + 1) % self.num_nodes
            token = self.nodes[next_node]
            self.nodes[next_node] = None

            # Break the loop if the token completes a full rotation
            if current_node == 0 and token is None:
```

```
            break

        # Move to the next node
        current_node = next_node


# Create an instance of TokenRing with 4 nodes
token_ring = TokenRing(4)

# Set the data for each node
token_ring.set_node(0, "Data 1")
token_ring.set_node(1, "Data 2")
token_ring.set_node(2, "Data 3")
token_ring.set_node(3, "Data 4")

# Run the token ring algorithm
token_ring.run()
```

```
PS D:\Distributed System Practicals> python -u "d:\Distributed System Practicals\Practical 5\
sample.py"
Node 0: Received token with data '['Data 1', 'Data 2', 'Data 3', 'Data 4']'
Node 0: Processing data 'Data 1'
Node 1: Received token with data 'Data 2'
Node 1: Processing data 'None'
Node 2: Received token with data 'Data 3'
Node 2: Processing data 'None'
Node 3: Received token with data 'Data 4'
Node 3: Processing data 'None'
Node 0: Received token with data 'Data 1'
Node 0: Processing data 'None'
```

# Experiment 6 Code:

## bully.py

```
class BullyAlgorithm:
    def __init__(self, nodeId, nodes):
        self.nodeId = nodeId
        self.nodes = nodes

    def startElection(self):
        highestNodeId = self.nodeId

        # Send "Coordinator Election" messages to all higher ID nodes
        for i in range(self.nodeId + 1, len(self.nodes) + 1):
            if i in self.nodes:
                print(f"Node {self.nodeId} sends Coordinator Election message to
Node {i}")

        # Wait for "OK" messages from all higher ID nodes
        for i in range(self.nodeId + 1, len(self.nodes) + 1):
            if i in self.nodes:
                print(f"Node {self.nodeId} waiting for OK message from Node {i}")

        # Check if there is a higher ID node that responded
        for i in range(self.nodeId + 1, len(self.nodes) + 1):
            if i in self.nodes:
                print(f"Node {self.nodeId} received OK message from Node {i}")
                highestNodeId = i

        # If there is a higher ID node, send "Coordinator" message to that
node
        if highestNodeId != self.nodeId:
```

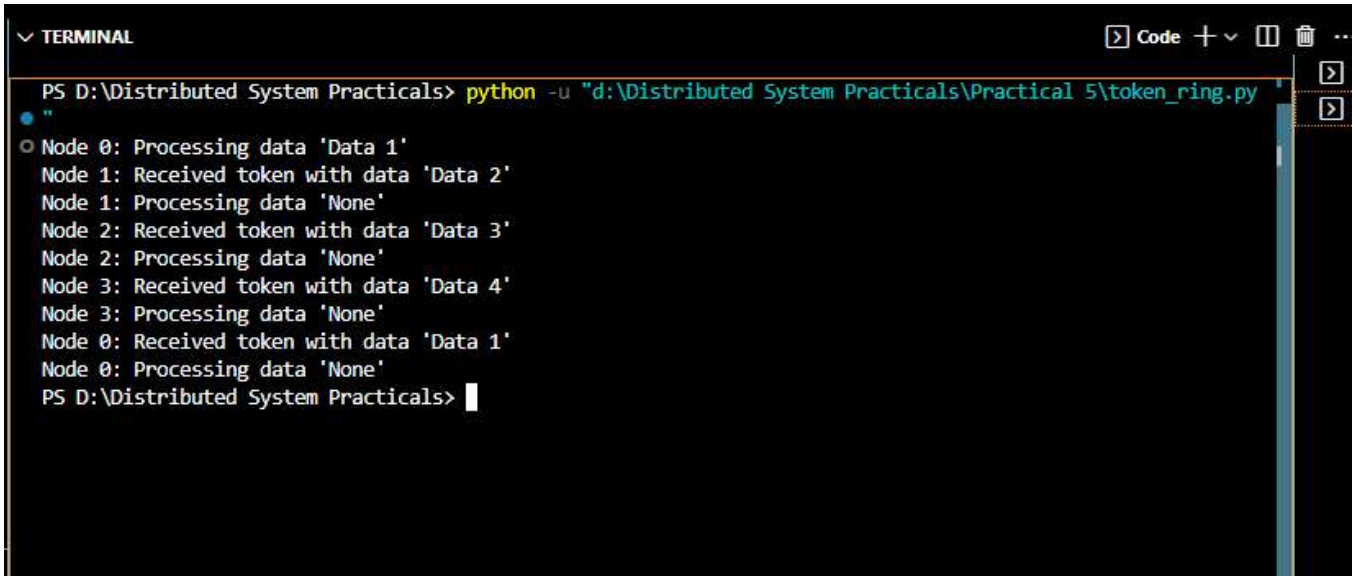
```
            print(f"Node {self.nodeId} sends Coordinator message to Node {highestNodeId}")

        # Otherwise, become the coordinator
        else:
            print(f"Node {self.nodeId} is the new coordinator")


# Create a list of node IDs
nodes = [1, 2, 3, 4, 5]

# Create an instance of BullyAlgorithm with a specific node ID and the list of nodes
bully = BullyAlgorithm(3, nodes)

# Start the election process
bully.startElection()
```



# Experiment 7 Code:

# (server code)

## Sample1.py

```
from flask import Flask, jsonify, request
app = Flask(__name__)
@app.route('/add', methods = ['POST'])
def add():
    data = request.get_json()
    a = data['a']
    b = data['b']

    result = a + b
    answer = jsonify({'result':result})
    return answer

if __name__ == '__main__':
    app.run()
```

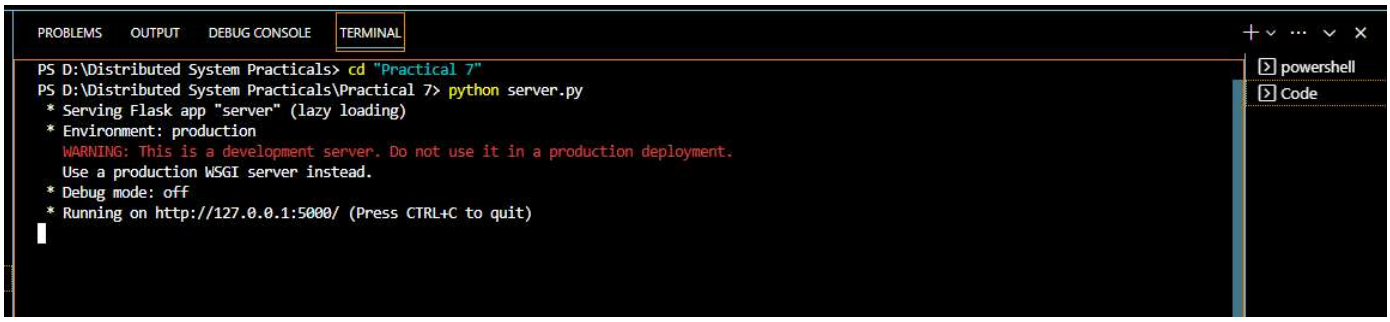
# (client code)

## Sample2.py

```
import requests
data = {
    'a':10,
    'b':20
}

response = requests.post('http://localhost:5000/add', json = data)
result = response.json()
print(result['result'])
```

## Output Screenshots



Steps to run the practical:
If necessary install the Flask module using: pip install flask
The first step is to run the server.py file on a separate terminal.

1. So, open a terminal and type:
   - python server.py (for windows)
   - python3 server.py (for linux)

2. Then open another terminal in the same directory in which both the code file lie. Type:
   - python client.py (for windows)
   - python3 client.py (for linux)